Problem Set 1

Applied Stats II

Due: February 11, 2026

Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the ith ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnoff CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs

poorly in small samples, but works well in a simulation environment. Write an `R` function that implements this test where the reference distribution is normal. Using `R` generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

As a hint, you can create the empirical distribution and theoretical CDF using this code:

```
  # create empirical distribution of observed data
  ECDF <- ecdf(data)
  empiricalCDF <- ECDF(data)
  # generate test statistic
  D <- max(abs(empiricalCDF - pnorm(data)))
```

```
######################
# load libraries
# set wd
# clear global .envir
######################

# remove objects
rm(list=ls())
# detach all libraries
detachAllPackages <- function() {
  basic.packages <- c("package:stats", "package:graphics", "package:grDevices", "package:utils", "package:datasets", "package:methods", "package:base")
  package.list <- search()[ifelse(unlist(gregexpr("package:", search()))==1, TRUE, FALSE)]
  package.list <- setdiff(package.list, basic.packages)
  if (length(package.list)>0)  for (package in package.list) detach(package, character.only=TRUE)
}
detachAllPackages()

# load libraries
pkgTest <- function(pkg){
  new.pkg <- pkg[!(pkg %in% installed.packages()[, "Package"])]
  if (length(new.pkg))
    install.packages(new.pkg, dependencies = TRUE)
  sapply(pkg, require, character.only = TRUE)
}

# here is where you load any necessary packages
# ex: stringr
# lapply(c("stringr"), pkgTest)

lapply(c(), pkgTest)

#set wd for current folder
setwd(dirname(rstudioapi::getActiveDocumentContext()$path))
getwd()
```

```r

######################
# Problem 1
######################
# ---------------------------------
# Kolmogorov Smirnov test function
# Reference distribution is Normal
# ---------------------------------
ks_normal_test <- function(data, mu = 0, sigma = 1, K = 100) {
  n <- length(data)
  #sort data
  x <- sort(data)
  #empirical CDF
  ECDF <- ecdf(x)
  empiricalCDF <- ECDF(x)
  #theoretical CDF (Normal)
  theoreticalCDF <- pnorm(x, mean = mu, sd = sigma)
  #KS statistic
  D <- max(abs(empiricalCDF - theoreticalCDF))
  #Kolmogorov-Smirnov CDF approximation
  ks_cdf <- function(d, K = 100) {
    sum(
      sqrt(2 * pi) / d *
        exp(-((2 * (1:K) - 1)^2 * pi^2) / (8 * d^2))
    )
  }
  #p-value (upper tail)
  p_value <- ks_cdf(D, K)

  return(list(
    D_statistic = D,
    p_value = p_value
  ))
}

#Simulation
set.seed(123)
#Generate Cauchy data
data <- rcauchy(1000, location = 0, scale = 1)
#Run KS test against Normal(0,1)
result <- ks_normal_test(data)
#Print results
print(result)
```

$$D = 0.1347281$$

$$p\text{-value} = 5.65 \times 10^{-29}$$

Question 2

Estimate an OLS regression in `R` that uses the Newton-Raphson algorithm (specifically `BFGS`, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`. Use the code below to create your data.

```r
######################
# Problem 2
######################

#OLS via BFGS vs lm()
#Data generation
set.seed(123)

data <- data.frame(
  x = runif(200, 1, 10)
)

data$y <- 0 + 2.75 * data$x + rnorm(200, 0, 1.5)

#Negative log-likelihood for OLS
neg_log_likelihood <- function(par, x, y) {
  beta0 <- par[1]
  beta1 <- par[2]
  sigma <- par[3]

  if (sigma <= 0) return(Inf)

  mu <- beta0 + beta1 * x

  -sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))
}


#Estimate using BFGS (quasi-Newton)
start_vals <- c(0, 1, 1)

bfgs_fit <- optim(
  par = start_vals,
  fn = neg_log_likelihood,
  x = data$x,
  y = data$y,
  method = "BFGS"
)

#Estimate using lm()
lm_fit <- lm(y ~ x, data = data)


#Compare results
```

```r
results <- data.frame(
  Parameter = c("Intercept", "Slope", "Sigma"),
  BFGS = c(
    bfgs_fit$par[1],
    bfgs_fit$par[2],
    bfgs_fit$par[3]
  ),
  LM = c(
    coef(lm_fit)[1],
    coef(lm_fit)[2],
    summary(lm_fit)$sigma
  )
)

print(results)
```

Parameter	BFGS	LM
Intercept	0.1391874	0.1391874
Slope	2.7266985	2.7266985
Sigma	1.4395450	1.4467966

Table 1: Comparison of OLS estimates using BFGS and `lm()`